Phase 1 — Project setup and planning

Goal: Prepare the development environment before writing the application.

Technologies: Git, GitHub, Node.js, npm, VS Code.

Tasks:

1. Create or inspect the existing Hello NSTUian repository.
2. Confirm the project requirements and documentation.
3. Configure the frontend and backend project structure.
4. Set up environment variables and .gitignore.
5. Define coding conventions and Git branching strategy.
6. Create a README containing setup instructions.

Deliverable: A clean, runnable repository with a documented structure.

Phase 2 — UI/UX design

Goal: Decide how the website will look and behave on mobile and desktop.

Technologies: Figma (optional), React, Tailwind CSS, custom CSS.

Design these screens first:

- Landing page with search and A–Z department cards.
- Department and session selection.
- Semester and course selection.
- Question list and question detail.
- Login and EduMail verification.
- Question submission and correction forms.
- Admin dashboard.

Deliverable: Approved responsive designs and reusable design components.

Do not spend too long polishing animations before the basic navigation works.

Phase 3 — Public frontend

Goal: Build the complete browsing experience using temporary sample data.

Technologies: React, Vite, React Router, Tailwind CSS.

Tasks:

1. Create the navigation bar, footer, and reusable glass cards.
2. Display departments in alphabetical order.
3. Implement department search.
4. Build session, semester, and course pages.
5. Add CT-1, CT-2, CT-3 and configurable sets.
6. Add semester/final question sections.
7. Create a question detail page with preview and download controls.
8. Test responsive behavior on mobile screens.

Deliverable: A navigable frontend with sample data. It is not yet connected to the production database.

Phase 4 — Database design

Goal: Build the persistent storage structure.

Technologies: PostgreSQL, Prisma ORM.

Tasks:

1. Create tables for departments, sessions, programs, semesters, and courses.
2. Create user and admin permission tables.
3. Create question-paper and version tables.
4. Create upload and correction request tables.
5. Create audit-log records.
6. Define foreign keys, indexes, and uniqueness constraints.
7. Write database migrations and development seed scripts.

Deliverable: A reproducible database that can be created from migrations.

Important: Confirm the actual department list, IDs, and academic curricula before inserting official data.

Phase 5 — Backend and API development

Goal: Connect the frontend to real database records.

Technologies: Node.js, Express.js, Prisma, Zod.

Tasks:

1. Configure the Express server.
2. Build department and course APIs.
3. Build session and semester APIs.
4. Build question listing and detail APIs.
5. Implement search, filtering, pagination, and error handling.
6. Validate input and verify database relationships.
7. Connect the React frontend to these APIs.

Deliverable: Public pages displaying real database data instead of hardcoded sample data.

Phase 6 — Authentication and EduMail verification

Goal: Allow users to contribute securely.

Technologies: Supabase Auth or another suitable identity provider, Express middleware.

Tasks:

1. Implement registration and login.
2. Verify eligible NSTU EduMail ownership.
3. Implement logout and session expiration.
4. Protect contribution endpoints.
5. Add account status checks.
6. Implement role and department permission middleware.
7. Configure rate limiting and secure authentication settings.

Deliverable: Public browsing remains open, while contribution features require verified authentication.

Before this phase is complete, confirm the actual NSTU EduMail domain and the available verification mechanism.

Phase 7 — Question upload and automatic publication

Goal: Implement the main contribution feature.

Technologies: Supabase Storage, Express.js, PostgreSQL, file-validation libraries.

Tasks:

1. Build the upload form.
2. Validate academic metadata.
3. Validate file type, content, and size.
4. Upload files to private storage initially.
5. Calculate a file hash and check for likely duplicates.
6. Automatically publish valid, genuinely new submissions.
7. Flag potential duplicates for review.
8. Create public previews and safe downloads.
9. Keep temporary and rejected files under a documented retention policy.

Deliverable: A working upload system with the agreed publication rules.

Phase 8 — Admin dashboard

Goal: Give administrators the tools to manage the platform.

Technologies: React, Express.js, PostgreSQL.

Tasks:

1. Build the admin overview.
2. Add department and course management.
3. Add question management.
4. Implement Super Admin permissions.
5. Implement Department Admin permissions.
6. Enforce department restrictions on every protected API.
7. Add admin assignment management.
8. Display relevant submission and audit information.

Deliverable: Administrators can manage records according to their assigned authority.

Phase 9 — Correction and approval system

Goal: Protect existing questions while allowing corrections.

Technologies: React, Express.js, PostgreSQL, private file storage.

Tasks:

1. Add the Request Correction interface.
2. Allow verified users to explain errors and propose replacement files.
3. Keep proposed files separate from published versions.
4. Build the admin review queue.
5. Implement approve, reject, and request-more-information actions.
6. Create a new official file version when a change is approved.
7. Preserve the old version and audit history.
8. Notify the contributor of the decision, where notification support is configured.

Deliverable: Existing questions cannot be modified directly by ordinary users.

Phase 10 — Testing and security verification

Goal: Confirm that the whole system works correctly.

Technologies: Vitest, React Testing Library, Supertest, Playwright.

Test at least:

- Public access to published questions.
- Rejection of unverified uploads.
- Automatic publication of eligible new questions.
- Duplicate detection.
- Rejection of unsafe files.
- Correction approval and rejection.
- Department Admin access restrictions.
- Super Admin global access.
- Audit history and file-version recovery.

- Mobile layouts and browser compatibility.

Deliverable: Automated test results and a list of resolved or remaining defects.

Phase 11 — Deployment

Goal: Make the application available online.

Technologies: A frontend host, a Node.js backend host, managed PostgreSQL, object storage, and a domain.

Tasks:

1. Deploy the frontend and backend.
2. Configure production environment variables.
3. Run database migrations.
4. Configure email verification.
5. Configure storage permissions.
6. Set up HTTPS and the production domain.
7. Configure logging, backups, and monitoring.
8. Test all public and administrative workflows in production.

Deliverable: A working production website.

Phase 12 — Maintenance and expansion

Goal: Keep the platform reliable as usage increases.

Tasks:

- Monitor errors and storage usage.
- Apply dependency and security updates.
- Test backups and restoration.
- Optimize search and database queries.
- Improve file preview performance.
- Add requested features only after evaluating their impact on existing workflows.

Deliverable: A maintainable system with a documented process for future changes.